Everplans QA Process Overhaul

Pitch Draft

Executive Summary

Everplans currently relies on 1 to 2 part-time QA contractors to absorb most of the final validation work across the product. As engineering output accelerates through AI assistance, the bottleneck has shifted downstream into QA, product clarification, environment setup, and release readiness. The core problem is not simply a lack of regression coverage. It is that the current operating model asks a very small amount of manual human validation to keep pace with a much larger volume of changing work.

This proposal recommends a top-to-bottom QA process overhaul: machine-readable product specifications, an evolving product canon, AI-assisted review of specifications, AI-assisted generation of targeted end-to-end coverage, explicit ownership of test authoring by engineers, stronger checkpoint gates, more stable environments, and a narrower but more valuable role for human QA centered on exploratory judgment and final UX confidence.

Current QA Bottlenecks

- Requirements and design often reach QA incomplete or unclear, so ambiguity is being discovered too late in the lifecycle.
- Testers sometimes enter unfamiliar areas without enough context and have to reconstruct product intent while trying to validate behavior.
- Ticket state, comment visibility, and retest readiness are not always clear, forcing QA to guess whether work is actually ready to come back through the queue.
- Scenario setup and test data creation take real time. When setup is missed or environment drift appears, QA ends up scrambling instead of validating.
- Environment inconsistency, including licensing, version mismatch, and production-parity issues, reduces confidence in both manual QA and automation.

These are not isolated annoyances. They are signs that too much discovery, coordination, and scenario creation is happening at the final QA stage instead of earlier in the process.

Core Thesis

If AI increases coding speed, Everplans should respond by redesigning how quality is produced. Specifications, test intent, scenario setup, and regression protection should be created earlier, reviewed earlier, and reused continuously. Quality work needs to move left. AI should be used not just to write code, but to make product specifications and operational knowledge machine-readable, transferable, and reviewable. Human QA should be reserved for the work humans are uniquely good at: exploratory testing, experiential judgment, and final confidence in the user experience.

Proposal Vision

The target state is a self-reinforcing system in which product specifications are reviewed for ambiguity before implementation starts, AI helps produce and maintain the product canon, engineers ship work together with automated coverage and setup guidance, and QA receives work only when the right validation artifacts are already in place. In that model, automation does more of the repetitive work, AI helps keep the system current, and human QA is freed to focus on the highest-value judgment calls.

How The System Works

1. Product and engineering define features in a format that both humans and AI can reliably read, with clear acceptance criteria, explicit UX expectations, and enough behavioral detail to reduce ambiguity before implementation begins.
2. AI reviews those specifications to surface critical edge cases, hidden ambiguity, and environment or setup requirements that might otherwise remain implicit until much later in the process.
3. The product canon is continuously produced and maintained as living documentation that captures workflows, expected behavior, scenarios, known edge cases, and test intent in a reusable form.
4. Developer utilities are paired with AI as reusable skills that can interact with test-environment databases and related systems to help provision test accounts and scenarios for engineers, end-to-end suites, and QA.
5. AI-assisted developer tooling, informed by the product canon and codebase history, helps generate end-to-end coverage, scenario fixtures, and targeted regression tests. Engineers still remain accountable for landing those assets as part of feature delivery.
6. Peer engineers review the tests alongside the implementation, using the product canon and ticket acceptance criteria as the source of truth rather than relying on tribal memory or ad hoc interpretation.
7. Human QA performs a focused manual pass centered on exploratory behavior, UX feel, and confidence that the delivered experience matches the spec and intent. QA also reviews whether the automated coverage is sufficient and sensible.
8. Tests are designed with a principle of being run often. That means easy ad hoc local execution, CI triggers based on pull requests and pushes to open pull requests, segmentation so higher-signal subsets can run more frequently, and full regression suites reserved for selected higher-risk release paths.
9. Test suites, scenario data, and the product canon are audited on a regular cadence, such as quarterly, so the system does not silently decay into flakiness, drift, or false confidence.

Self-Reinforcing

- Every better specification improves AI-generated documentation and test suggestions.
- Every bug can become both a regression test and a clearer canon entry.
- Every new scenario fixture reduces future setup scramble for QA.
- Every reviewed AI-generated test improves the team’s prompt patterns, guardrails, and confidence in the system.

Recommended Operating Model

- Product and PM own clearer acceptance criteria, scenario expectations, and explicit definitions of done before work reaches QA.
- Engineers own shipping the code together with the regression protection, scenario fixtures, and documentation updates needed for others to verify it.
- Engineering reviewers own validating that tests and canon updates reflect the intended behavior, not merely that the implementation appears reasonable on first pass.
- QA owns exploratory testing, experiential judgment, risk-based scrutiny, and a final pass on whether the automated strategy is covering the right things.
- A release or process owner should ideally own environment health, fixture reliability, and visibility into whether tickets are truly ready for QA or retest, though Everplans may need to absorb that responsibility into an existing role if staffing is limited.

Delivery Model

- Recommended structure: one strong stateside technical or process lead paired with a Portugal-based implementation pod made up of one PM or process coordinator and one to two engineers or test automation specialists.
- Best offshore scope: building the product-canon workflow, standing up the initial end-to-end suites, configuring the supporting process infrastructure, building AI-assisted tooling and reusable skills, and carrying periodic audits of the system over time.
- Best onshore scope: architecture decisions, risk prioritization, acceptance-criteria quality bar, release policy, and alignment with product leadership and senior engineering stakeholders.

Phased Rollout

1. Assessment and process map: quantify where QA time is actually going today, including requirement ambiguity, setup time, retest churn, environment mismatch, and repetitive validation.
2. AI-assisted specification review: introduce a product-spec review step, ideally integrated with Jira, where AI evaluates tickets, subtasks, acceptance criteria, and linked designs for ambiguity, missing edge cases, missing environment or data-setup requirements, and unclear definitions of done before implementation begins.
3. Foundation: define the canon format, improve ticket readiness criteria, standardize retest visibility, create stable scenario fixtures and environment checklists, and make AI spec-review output part of normal backlog refinement.
4. Pilot: stand up a small, high-value end-to-end suite for the most critical workflows, with CI triggers, local ad hoc runs, explicit failure diagnostics, and a lightweight manual QA test-plan template attached to the work.
5. Checkpoint gates: define automated checkpoints for ticket readiness, developer-complete, and QA-ready states. When engineering marks work complete, the system should confirm that required automated tests exist, the manual QA test plan is present, scenario and setup needs are documented, and canon updates or behavior notes have been captured.
6. Scale: integrate AI-assisted generation and maintenance into the delivery workflow, expand suite coverage selectively, and establish quarterly audits for suite quality, checkpoint compliance, and canon freshness.